

```
// VOICE LAYER. Uses the speech
features already built into the
phone's browser
// (free, no extra accounts):
SpeechRecognition = ears,
speechSynthesis = mouth.

import { JarvisError } from
"./errors.js";

// ----- pure helpers (easy to
test) -----

export function cleanForSpeech(text)
{
    return String(text || "")
        .replace(/``[\s\S]*?``/g, " ")
        .replace(/\s*([^\s])\s*/g, "$1")
        .replace(/[*`#>~]/g, "")
        .replace(/\s+/g, " ")
        .trim();
}

// Long utterances get cut off in
some browsers, so speak sentence-
```

```

sized pieces.
export function splitForSpeech(text,
maxLen = 180) {
  const sentences =
text.match(/[^.!?]+[.!?]*\s*/g) ||
[];
  const parts = [];
  let current = "";
  for (const s of sentences) {
    if (current && (current +
s).length > maxLen) {
parts.push(current.trim()); current =
""; }
    current += s;
    while (current.length > maxLen *
2) { // one giant sentence: cut at a
space
      let cut = current.lastIndexOf("
", maxLen);
      if (cut < 1) cut = maxLen;
      parts.push(current.slice(0,
cut).trim());
      current = current.slice(cut);
    }
  }
  if (current.trim())
parts.push(current.trim());
  return parts;
}

```

```

function mapSttError(code) {
  switch (code) {
    case "not-allowed":
    case "service-not-allowed":
return "mic_denied";
    case "audio-capture": return
"audio_capture";
    case "no-speech": return
"no_speech";
    case "network": return
"stt_network";
    case "aborted": return "aborted";
    default: return "stt_failed";
  }
}

// ----- ears -----

export class Listener {
  constructor({ lang = "en-US", win =
globalThis } = {}) {
    this.lang = lang;
    this.win = win;
    this.rec = null;
    this.aborting = false;
  }

  static isSupported(win =

```

```
globalThis) {
    return
    Boolean(win.SpeechRecognition ||
win.webkitSpeechRecognition);
}

    // Resolves with the transcribed
text. Rejects with a JarvisError.
    listen({ onInterim } = {}) {
        const Ctor =
this.win.SpeechRecognition ||
this.win.webkitSpeechRecognition;
        if (!Ctor) return
Promise.reject(new
JarvisError("stt_unsupported"));
        this.abort();
        this.aborting = false;
        return new Promise((resolve,
reject) => {
            const rec = new Ctor();
            this.rec = rec;
            rec.lang = this.lang;
            rec.continuous = false;
            rec.interimResults = true;
            rec.maxAlternatives = 1;
            let finalText = "";
            let interimText = "";
            let settled = false;
            const done = (fn, value) => {
```

```

        if (settled) return;
        settled = true;
        if (this.rec === rec)
this.rec = null;
        fn(value);
    };
    rec.onresult = (e) => {
        let f = "", i = "";
        for (let k = 0; k <
e.results.length; k++) {
            const r = e.results[k];
            if (r.isFinal) f +=
r[0].transcript; else i +=
r[0].transcript;
        }
        finalText = f;
        interimText = i;
        if (onInterim) onInterim((f +
i).trim());
    };
    rec.onerror = (e) =>
done(reject, new
JarvisError(mapSttError(e.error),
String(e.error)));
    rec.onend = () => {
        const text = (finalText ||
interimText).trim();
        if (this.aborting)
done(reject, new

```

```
JarvisError("aborted"));
    else if (text) done(resolve,
text);
    else done(reject, new
JarvisError("no_speech"));
};
try {
    rec.start();
} catch (e) {
    done(reject, new
JarvisError("stt_failed", String(e),
e));
}
});
}

// Stop listening and use what was
heard so far.
stop() { if (this.rec)
this.rec.stop(); }

// Stop listening and throw the
audio away.
abort() {
    if (this.rec) { this.aborting =
true; this.rec.abort(); }
}
}
```

```
// ----- mouth -----

export class Speaker {
  constructor({ lang = "en-US", win =
globalThis } = {}) {
    this.lang = lang;
    this.win = win;
    this.token = 0;
    this._finish = null;
  }

  static isSupported(win =
globalThis) {
    return
Boolean(win.speechSynthesis &&
win.SpeechSynthesisUtterance);
  }

  // iPhones only allow speech that
  was "unlocked" by a tap. Call this
  inside the tap handler.
  unlock() {
    try {
      if
(!Speaker.isSupported(this.win))
return;
      const u = new
this.win.SpeechSynthesisUtterance("")
;

```

```
        u.volume = 0;

this.win.speechSynthesis.speak(u);
    } catch { /* not critical */ }
}

speak(text) {
    if
(!Speaker.isSupported(this.win))
return Promise.reject(new
JarvisError("tts_unsupported"));
    const parts =
splitForSpeech(cleanForSpeech(text));
    if (!parts.length) return
Promise.resolve();
    this.stop();
    const synth =
this.win.speechSynthesis;
    const myToken = ++this.token;
    return new Promise((resolve,
reject) => {
        let i = 0;
        this._finish = resolve;
        const next = () => {
            if (myToken !== this.token)
return;
            if (i >= parts.length) {
this._finish = null; resolve();
return; }

```

```

        const u = new
this.win.SpeechSynthesisUtterance(par
ts[i++]);
        u.lang = this.lang;
        u.onend = next;
        u.onerror = (e) => {
            if (myToken !== this.token)
return;
            this._finish = null;
            if (e.error === "canceled"
|| e.error === "interrupted")
resolve();
            else reject(new
JarvisError("tts_failed",
String(e.error)));
        };
        synth.speak(u);
    };
    next();
});
}

stop() {
    this.token++;
    const finish = this._finish;
    this._finish = null;
    try { if
(Speaker.isSupported(this.win))
this.win.speechSynthesis.cancel(); }

```

```
catch { /* ignore */ }  
    if (finish) finish();  
}  
}
```